MYSQL Query Flow

What is storage engine ?

In mysql the storage engine will holds the actual data and say that it handle the major storage part .

InnoDB handle :
table data
indexes
transactions
locking
redo log
crash recovery

SQL
↓
MySQL Server Layer
↓
InnoDB
↓
Memory + Disk

The journey of SELECT Statement .

```
SELECT name
FROM employee
WHERE id = 101;
```

Application
↓
Connection
↓
Authentication
↓
Parser
↓
Optimizer
↓
Execution
↓
InnoDB
↓
Buffer Pool
↓
Find row
↓
Return data
↓
Application

Buffer Pool :

RAM

```
┌─────────────────────────────┐
|      InnoDB Buffer Pool     |
|                             |
| employee data               |
| customer data               |
| indexes                     |
| other frequently used pages |
└─────────────────────────────┘
```

Note : Mysql frequently used data putting into the RAM to avoid the data reading from disk if we run the query to frequently .

If data found in RAM :

SELECT id=101
↓
Buffer Pool
↓
Found!
↓
Return

If data not found in RAM :

SELECT
↓
Buffer Pool
↓
Not found
↓
Disk se page read
↓
Buffer Pool load
↓
Data return

Next is INSERT :

INSERT INTO employee
(id, name, salary)
VALUES
(101, 'Rahul', 50000);

Application
↓
MySQL
↓
Parser
↓
Optimizer
↓
Execution
↓
InnoDB
↓
Buffer Pool

Note : Innodb modify the relevant data in memory . & not immediately put into the .ibd file .

Innodb is use the redo log file .

What is redo log ?

INSERT
↓
Memory changes
↓
Redo Log
↓
Commit

Note : Redo log basically recovery ke liye record rakhta hai ki database pages mein kya changes kiye gaye.

INSERT Rahul
↓
Server crash (note : if data are in ram then data will gone)

So :

RAM
↓
Redo Log
↓
COMMIT
Then :
Restart
↓
Redo Log
↓
Recover changes

But when data are going to .idb file

INSERT
↓
Buffer Pool page modified
↓
Page becomes "dirty"
↓
Redo log records change
↓
Commit durability handled
↓
Later background flushing
↓
.ibd data file updated

How update query will work internally :

UPDATE employee SET salary = 60000 WHERE id = 101;

UPDATE
↓
Parser
↓
Optimizer
↓
Find row/index
↓
InnoDB
↓
Buffer Pool
↓
Modify page
↓
Redo Log
↓
Commit

And later :

Dirty Page
↓
Disk flush
↓
.ibd

Delete :

DELETE FROM employee
WHERE id = 101;

DELETE
↓
Find row
↓
Modify InnoDB page
↓
Redo Log
↓
Commit
↓
Later disk flush

Now we have to see it daigrametically :

```
      APPLICATION
          |
          | SQL
          ↓
  ┌─────────────────┐
  |  MySQL Server   |
  |                 |
  | Connection      |
  |     ↓           |
  | Parser          |
  |     ↓           |
  | Optimizer       |
```

```
|    ↓                  |
| Execution             |
└──────────┬────────────┘
           |
           ↓
     ┌──────────┐
     | InnoDB   |
     └────┬─────┘
          |
┌─────────┴─────────┐
↓         ↓
BUFFER POOL    REDO LOG
|                  |
|                  |
↓              ↓
RAM            DISK
|
↓
Dirty Pages
|
↓
Background Flush
|
↓
DATA FILE
(.ibd)
```

SELECT vs INSERT/UPDATE/DELETE

SELECT
↓
Find data
↓
Buffer Pool
↓
If missing → Disk read
↓
Return result

INSERT
↓
Create/change data
↓
Buffer Pool
↓
Redo Log
↓
Commit
↓
Later flush to data file

UPDATE
↓
Find existing data
↓
Modify
↓
Buffer Pool
↓
Redo Log
↓
Commit
↓
Later flush

DELETE
↓
Find data
↓
Modify/delete logically
↓
Buffer Pool
↓
Redo Log
↓
Commit
↓
Later flushing/purge

Simple in Hindi Sort Language .

1. Client query bhejta hai
↓
2. Connection receive karta hai
↓
3. Parser SQL samajhta hai
↓
4. Optimizer plan choose karta hai
↓
5. PRIMARY KEY index use hota hai
↓
6. Relevant B+Tree pages locate hote hain
↓
7. Page Buffer Pool mein check hota hai
↓
8. Agar RAM mein nahi → disk se read
↓
9. Page Buffer Pool mein aata hai
↓
10. Row locate hoti hai
↓
11. Result client ko return

Created By : Ajit Yadav (MSSQL , MYSQL , PostgreSQL DBA)